# Lab 12. Automate App Deployment to an OKE Using DevOps CI/CD

## Overview

The Oracle Cloud Infrastructure (OCI) DevOps service is an end-to-end, continuous integration and continuous delivery (CI/CD) platform for developers.

In this lab, you will:

a. Create DevOps deployment pipeline and deploy stage.
b. Create Trigger Deployment Stage in build pipeline.
c. Automate Sample Web Application deployment to OKE cluster.

### Prerequisites

- You are signed into your Oracle Cloud Infrastructure (OCI) account using your credentials.
- You must complete the following labs before you perform tasks for this practice:
  - *Create Docker image for a web application using Dockerfile*.
  - *Create and work with OCIR repository*.
  - *Deploy a load-balanced Web application on an OKE cluster using Kubectl*.
  - *Work with code repositories in OCI DevOps project*.
  - *Create and set up artifacts and environments in DevOps project*.

### Assumptions

- A pre-created OKE cluster **OKE-Cluster** is available in the your compartment.
- An OCIR repo `<region-key>-aiops-lab04-ocir-1/webapp_<userID>`.
- You have an auth token `AIOPS-LAB04-AT-01`.
- You have a namespace `ns-<userID>` and Kubernetes Secret `ocir-secret-<userID>` created

### Sequence 1. Create Deployment Pipeline and Deploy Stage

DevOps service includes predefined stages, which could be readily used in a deployment pipeline:

- **Deploy to a Kubernetes cluster**: Uses the built-in Kubernetes rolling update strategy
- **Deploy to an instance group**: Releases update incrementally to the instance group. You can specify the maximum instances that can be offline at one time. This type supports automatic rollbacks.
- **Deploy to Functions**: Uses the built-in Functions update strategy In this lab, you will create DevOps deployment pipeline and deploy stage.

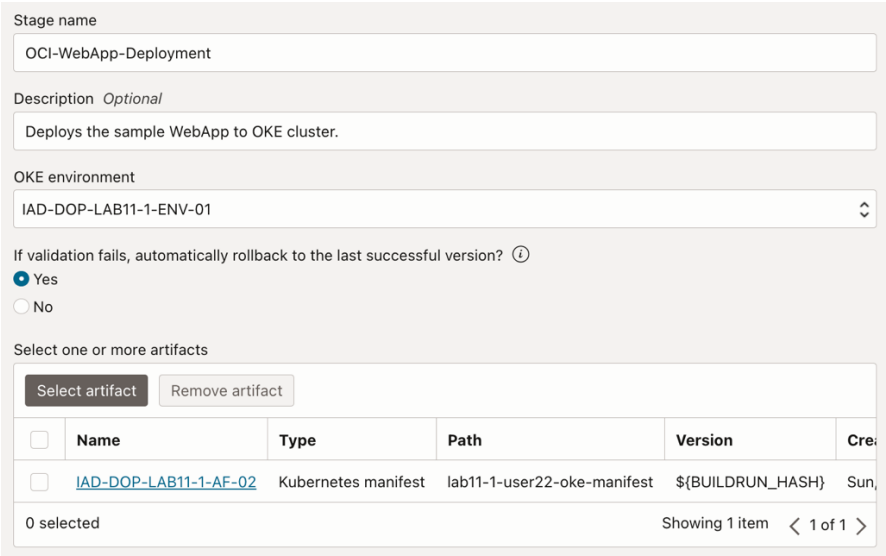
1. Open the project **IAD-DOP-LAB05-1-DP-01-*<userID>***. (IAD-DOP-LAB05-1-DP-01-USER16)
2. Click **Deployment Pipelines** from the left menu to navigate to the Deployment Pipelines page.
3. Click **Create pipeline** and fill the form with the following values:
   a. **Name:** `IAD-DOP-LAB12-1-DPL-01`
   b. **Description:** `This is Deployment pipeline for Lab12.`

c. Click **Create pipeline.** The **Pipeline** tab will open.

4. To add a stage to the Deployment pipeline, click the **+** icon and click **Add stage.**
   a. Select **Apply manifest to your kubernetes cluster** as stage type and click **Next.** Fill the form with the following values:
      1) **Stage name**: `OCI-WebApp-Deployment`

      2) **Description**: `Deploys the sample WebApp to OKE cluster.`

      3) **Environment**: Select the environment **IAD-DOP-LAB11-1-ENV-01**
      4) Under **Select one or more artifacts** field, click **Select Artifact.** This opens the window to add Kubernetes Manifest type artifacts.
         - Select **IAD-DOP-LAB11-1-AF-02** and click **Save changes**.




      5) **Override Kubernetes namespace:** Leave this field blank. The namespace is specified in the `HelloWorld-lb.yaml` file which is `ns-<userID>`.
      6) **If validation fails, automatically roll back to the last successful version?:**

         Select **Yes** to automatically roll back to the last successful version.

      7) Click **Add**. You will notice a stage with name **OCI-WebApp-Deployment (Deploy OKE: Rolling)** added.

## Sequence 2. Create a Trigger Deployment Stage in Build Pipeline

In this sequence, you will create a Trigger Deployment Stage within the build pipeline that triggers the Deployment pipeline to deploy the application based on the output artifacts from the build pipeline execution.

1. Navigate to the Build pipeline **IAD-DOP-LAB11-1-BPL-01** in your DevOps project.
2. On the **Build Pipeline** tab, click the **+** icon at the **bottom** of the Push WebApp Artifacts (Deliver Artifacts) box and click **Add stage**.
   a. Select **Trigger Deployment** as stage type from the optional section and click **Next**. Fill the

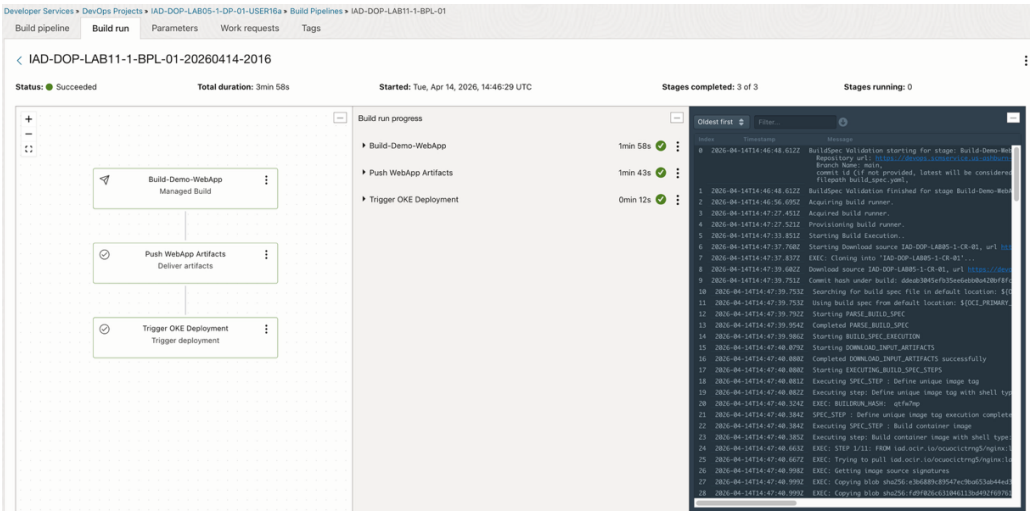
form with the following values:

1) **Stage name**: `Trigger OKE Deployment`
2) **Description**: `This triggers the IAD-DOP-LAB12-1-DPL-01 Deployment pipeline stages.`
3) Click **Select Deployment Pipeline**. Select **IAD-DOP-LAB12-1-DPL-01** and click **Save changes**.
4) Enable the **Send build pipelines Parameters** option.
5) **Artifacts used in the deployment pipeline** auto-populate with the Kubernetes manifest type artifact **IAD-DOP-LAB11-1-AF-02**.
6) Click **Add**. You will notice a stage with name **Trigger OKE Deployment (Trigger deployment)** added.

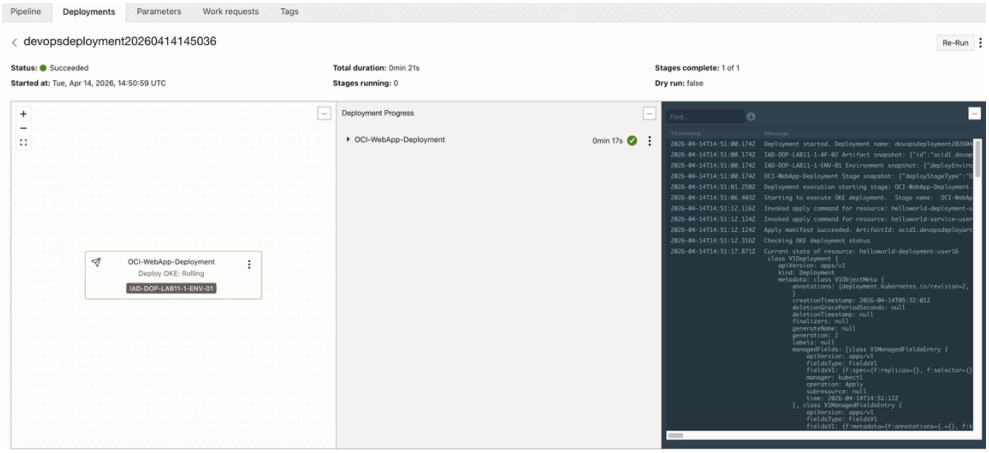
## Sequence 3. Automate Sample Web Application Deployment to OKE Cluster

In this sequence, you will run the Build pipeline to execute all its stages in sequence and populate the artifact and container registry with the manifest and Docker image, respectively. The successful execution of the Build pipeline will trigger the Deployment pipeline, which uses the output artifacts and applies them to the target environment, which in this case is an OKE cluster.

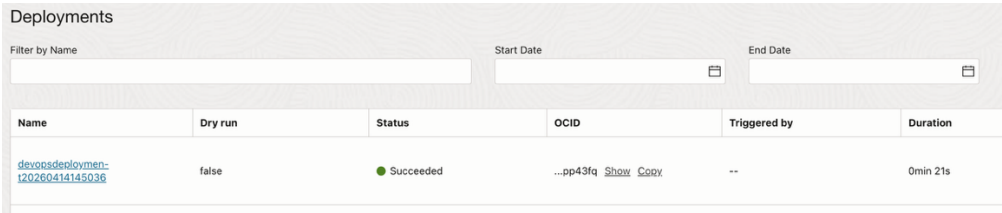
1. On the Build Pipeline page **IAD-DOP-LAB11-1-BPL-01**. Click the **Start Manual Run** button. The Start Manual Run page opens.
   a. OCI assigns your build a Build run name.
   b. Click **Start manual run** at the bottom left.
2. You will reach the **Build pipeline** tab. Observe that all the build stages are listed here. Build stages will execute sequentially. You can observe the logs for each stage in the right window.




3. Once the Trigger OKE Deployment stage completes, click **Deployments** from the left menu under your DevOps project **IAD-DOP-LAB05-1-DP-01-<*userID*>** to navigate to the Deployments page.




You will observe deployment listed here that got automatically kicked-off and is either in an **In-progress** or **Succeeded** state.




Further, when you click the deployment name, you will reach the **Deployments** tab. Under the **Deployments** tab, you can see the logs and additional details for the Deployment pipeline run.

4. Because the deployment is successful, let's now try to access the application using the External (or Public) endpoint that is the HelloWorld Service Load Balancer IP.
5. Open the terminal in your Jupyter lab **and switch to root user.**

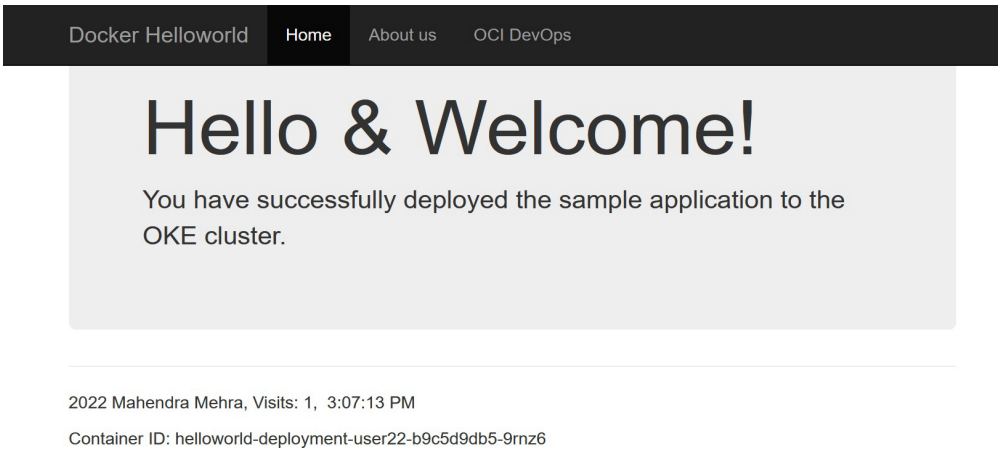
```
$ sudo su -
kubectl get svc -n ns-<userID>
```

For example,

```
kubectl get svc -n ns-user16
```

You will observe the **EXTERNAL-IP** listed in the output.

6. Launch a Web browser and enter the EXTERNAL-IP address into the browser's address bar to access the application. Once the request is processed, you'll see the following webpage:




Every time you run a build pipeline, these artifacts will be generated and stored in the container and artifact registry with a unique string to identify them. In case of a build failure, these artifacts are used to roll back to last successful version.

**Important Note**: **Do not delete any artifacts and resources created in this lab because they will be required in the upcoming labs.**

Congratulations! You have successfully deployed a Web Application to an OKE cluster using OCI DevOps Build and Deployment pipelines.